

Informe Tarea 2 Sistemas Operativos

Informe de Proyecto



Integrantes:

José González Aguayo

Gonzalo Quilodrán Neira

Joaquín Sandoval Reyes

Universidad de Concepción

2 de diciembre de 2025

Índice

1. Resumen	2
2. Introducción	2
3. Parte I: Sincronización con Barrera re-utilizable	2
3.1. <code>barrera.h</code>	2
3.2. <code>barrera.c</code>	3
3.3. <code>main.c</code>	4
3.4. Prueba y Resultados	6
4. Parte II: Simulador simple de Memoria Virtual	8
4.1. <code>funcion.h</code>	8
4.2. <code>funcion.c</code>	8
4.3. <code>sim.h</code>	9
4.4. <code>sim.c</code>	9
4.5. <code>main.c</code>	10
4.6. Prueba y Resultados	11
5. Conclusiones	12

1. Resumen

Los sistemas operativos, aquellos que permiten una administración de recursos además de permitir un nivel de abstracción para facilitar la interacciones del usuario. Relacionado a estos, existen dos conceptos fundamentales: la sincronización de procesos y la administración de memoria. Estos son fundamentales para evitar errores catastróficos. Para un ingeniero civil Informático, es fundamental poder aplicar y entender estos temas.

2. Introducción

En el presente documento, recopilamos el paso a paso de la construcción de dos códigos, el primero siendo una barrera re-utilizable la cual permite sincronizar un número N de hebras. El segundo trata de un simulador de administración de memoria virtual, para poder apreciar de mejor forma como funciona la paginación.

La finalidad de realizar ambos códigos es la de entender y comprender el funcionamiento de herramientas como los **Monitores**, el uso de las herramientas de **pthread**, la **paginación**, la lectura de **direcciones virtuales** y el como se traduce a **direcciones físicas**, entre otras herramientas que se usaron y que se explicaran a continuación.

Todo lo desarrollado, se encuentra en el siguiente repositorio de GitHub:

REPOSITORIO TAREA 2 SISTEMAS OPERATIVOS

3. Parte I: Sincronización con Barrera re-utilizable

El objetivo planteado para este apartado era el diseñar una barrera primitiva que sea re-utilizable y que permita sincronizar múltiples hebras concurrentes, usando la biblioteca **pthread**.

La idea es que el usuario ingrese un número N de hebras y un número E de etapas, donde las hebras harán una tarea. Primero se debería generar esas N hebras, empezando cada una en la etapa 0, cuando una hebra termine su tarea, no puede avanzar y se queda esperando hasta que la ultima hebra termine, ahí recién pasando a la siguiente etapa. Cada vez que se cambie de etapa, el monitor debe resetearse automáticamente, para que se puedan reutilizar.

Para el monitor, se usaron las siguientes bibliotecas: **stdio.h**, **stdlib.h**, **pthread.h**, **unistd.h**, y **time.h**. Además el código, lo dividimos en tres archivos principales:

3.1. `barrera.h`

Este archivo es el esqueleto del monitor, ya que contiene las bases de **barrera_t**, definiendo un **struct** donde se crean las variables principales:

- **n**: El número total de hebras que recibirá el monitor.
- **count**: Variable contador que indica cuántas hebras han llegado a la barrera.
- **etapa**: Variable que representa en qué etapa se encuentra trabajando una hebra.

- **cond:** Variable de tipo `pthread_cond_t`. Esta condición permite que las hebras esperen hasta que todas hayan llegado a la barrera.
- **mutex:** Variable de tipo `pthread_mutex_t`. Es la exclusión mutua que protege la sección crítica, en este caso las variables compartidas de la barrera.

Luego, en el archivo se declaran las tres funciones principales que tendrá la barrera, para especificar al compilador que esas funciones existen, aunque no sea en este archivo.

```

1  #ifndef BARRERA_H
2  #define BARRERA_H
3  #include <pthread.h>
4
5
6  /*
7  Estructura básica de la barrera, con las variables principales. Estas son a las que todas las hebras
8  (que estén ligadas a la misma barrera) podrán acceder.
9  */
10 typedef struct{
11     int n; //numero total de hebras
12     int count; //numero de hebras que han llegado a la barrera
13     int etapa; //etapa donde opera una hebra
14     pthread_cond_t cond;
15     pthread_mutex_t mutex;
16 } barrera_t;
17
18 /*
19 Aquí declaramos las funciones que tendrá barrera. Si esto no se hace, el compilador dará error.
20 */
21 void barrera_init(barrera_t* b, int N);
22 void barrera_wait(barrera_t* b);
23 void barrera_destroy(barrera_t* b);
24
25 #endif

```

Figura 1: foto del código `barrera.h`

3.2. `barrera.c`

En este código se crean las funciones principales de la barrera. Primero con **`barrera_init`**, la cual se encarga de inicializar la barrera. Al ser llamada se piden dos argumentos, el puntero de una `barrera_t` `b` y un número entero `N`. Dentro de la función, se inicializan las cinco variables de la barrera `b`, igualando **`count`** y **`etapa`** a cero, **`n`** dándole el valor de `N`, y inicializando el mutex y `cond`, estas últimas con las funciones **`pthread_mutex_init`** y **`pthread_cond_init`**.

La siguiente función es **`barrera_wait`**, la cual al ser llamada se le debe entregar un puntero de `barrera_t`. Esta es llamada por una hebra, donde primero bloquea el mutex para que ninguna otra hebra pueda acceder o modificar las variables globales de la barrera (ya que recordemos que las variables de la barrera, pueden ser modificadas por cualquier hebra), luego crea una variable local **`etapa_local`** que será igual a la variable **`etapa`** de la barrera, además a **`count`** se le suma uno. Luego, si `count` es menor a `n` (o sea, el número de hebras que han llegado a la barrera es menor al número total de hebras que la barrera espera), entonces la hebra entra a un `while` infinito mientras la variable `etapa` de barrera sea igual a la variable de etapa local, en este ciclo, con la función **`pthread_cond_wait`** la hebra se "duerme", soltando el mutex, permitiendo que otra hebra pueda entrar a la barrera, tome y bloquee el mutex y siga los mismos pasos de la hebra anterior. Esto seguirá una y otra vez, hasta que llegue la última hebra, que al entrar a la barrera, aumenta la variable `count` para que quede del mismo valor que `n`, tras esto, esta hebra

pasa a la otra condicion (else), donde aumenta en uno la variable **etapa** (se passa a la siguiente etapa), a la variable **count** la resetea igualandola a cero (permitiendo que la barrera pueda ser reutilizada para la siguiente etapa) y con la función **pthread_cond_broadcast** despierta a todas las hebras que estaban dormidas, saliendo del ciclo while, y luego cada hebra desbloquean el mutex y salen de la función.

Por ultimo, se crea la función **barrera_destroy**, que se encarga de destruir el mutex y cond (condición), esto con las funciones **pthread_mutex_destroy** y **pthread_cond_destroy**.

Con estas tres funciones, ya tenemos lista nuestra barrera/monitor re-utilizable.

```

1  #include <stdio.h>
2  #include "barrera.h"
3
4
5  /*
6  Esta función se encarga de inicializar la barrera, asignando valores a las variables
7  del struct y de inicializar el cond y el mutex.
8  */
9  void barrera_init(barrera_t* b, int N){
10     b->count = 0;
11     b->etapa = 0;
12     b->n = N;
13     pthread_mutex_init(&b->mutex, NULL);
14     pthread_cond_init(&b->cond, NULL);
15 }
16
17 /*
18 Esta es la función principal de la barrera, wait. Primero se cierra el mutex y se guardan en variables locales
19 el valor de la etapa y el contador aumenta en 1.
20 Luego si el valor de contador de la barrera es menor que el numero de hebras representado por N, la hebra se queda en espera (se duerme).
21 En caso contrario, a la variable etapa de la barrera se aumenta en 1 y el contador se devuelve a 0, y con broadcast despiertan
22 todas las hebras que hay. Tras esto, se desbloquea el mutex.
23 */
24 void barrera_wait(barrera_t* b){
25     pthread_mutex_lock(&b->mutex);
26     int etapa_local = b->etapa;
27     b->count = b->count + 1;
28
29     if(b->count < b->n){
30         while(b->etapa == etapa_local){
31             pthread_cond_wait(&b->cond, &b->mutex);
32         }
33     }else{
34         b->etapa = b->etapa + 1;
35         b->count = 0;
36         pthread_cond_broadcast(&b->cond);
37     }
38     pthread_mutex_unlock(&b->mutex);
39 }
40
41 /*
42 Esta función se encarga de destruir tanto el mutex, como la condicion cond. Esto es indispensable.
43 */
44 void barrera_destroy(barrera_t* b){
45     pthread_mutex_destroy(&b->mutex);
46     pthread_cond_destroy(&b->cond);
47 }

```

Figura 2: foto del código barrera.c

3.3. main.c

Primero en este código, se crea otro struct para guardar las variables de la hebra, **dHebra**, en esta hay un entero que representa el id de la hebra, un entero E que representa la etapa de la hebra, y el puntero de una barrera_t que llamaremos b.

Luego, se creo una función **nRandom**, que genera un número n random, esto se utilizara más adelante.

Posterior, esta la función **trabajo_hebra**, la cual recibe un argumento (arg), que es el struct de datos de una hebra. Se crean tres variables locales, una del id de la hebra, de la etapa de la hebra y una que guarde el punteo de la barrera (los valores de cada una son sacados desde dHebra). Luego, en un ciclo for, que seguira mientras la variable i sea menor que el valor de la etapa, la hebra se duerme con **usleep**, dandole un valor aleatorio que obtenemos de la función **nRandom()**, luego imprime en terminal que "[id hebra] esperando en etapa [número de etapa]", posterior llama a **barrera_wait** entregando a la función el puntero de la barrera con la que trabajamos. Tras salir del monitor se imprime por terminal "[id hebra] paso barrera en etapa [número de etapa]", tras esto el ciclo suma uno a la variable i y continua. La función devuelve un valor NULL.

Por ultimo, tenemos la función principal, el main. Primero con **srand(time(NULL))** se genera una semilla distinta en cada ejecución para que la función nRandom genere números distintos en cada ocasión. Luego, se declaran dos variables: N que será el número de hebras y E que es el número de etapas, después se imprime en pantalla el siguiente mensaje: "Numero de hebras y Numero de Etapas" luego con **scanf()** el usuario ingresa dos valores a su elección. Se crea una instancia de **barrera_t** llamada **b**, y se inicializa con **barrera_init**. Tras esto se genera un arreglo con N espacios de **dHebras** llamado **datos**, y en un ciclo for que se repetirá mientras *i* sea menor que N, en cada iteración se inicializa cada **dHebras**. Con los datos de cada hebra ya inicializados, se crea otro arreglo de extensión N de **pthread_t**, este es llamado **hebras**, y en un ciclo for se inicializa cada hebra con la función **pthread_create**, entregando primero la dirección de memoria de *hebras[i]*, un valor NULL, se especifica qué función seguirá la hebra (en este caso se entrega la función **trabajo_hebras**) y la dirección de memoria de los datos de la hebra ligados al arreglo **datos[i]**.

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <pthread.h>
4  #include <unistd.h>
5  #include "barrera.h"
6  #include <time.h>
7
8  /*Para compilarlo se necesita el siguiente comando
9   gcc main.c barrera.c -lpthread -o programa
10  y luego ./programa.exe
11  */
12
13
14  /*
15  Este struct sirve unicamente para pasar más de un dato que debería tener
16  cada hilo. Ya que teoricamente no tienen un numero fijos de hilos ni de
17  etapas.
18  */
19  typedef struct {
20      int id;
21      int E;
22      barrera_t* b;
23  } dHebra;
24
25  /*
26  Esta funcion genera un numero alatorio.
27  */
28  int nRandom(){
29      int n = rand()%1000;
30      return n;
31  }
32
33  /*
34  Esta funcion se encarga de simular que la hebra haga algo.
35  */
36  void* trabajo_hebra(void* arg){
37      dHebra* d = (dHebra*)arg;
38      int id = d -> id;
39      int Etapa = d -> E;
40      barrera_t* barrera = d -> b;
41
42      for(int i=0; i<Etapa; i++){
43          usleep(nRandom());
44          printf("%d esperando en etapa %d.\n", id,i);
45          barrera_wait(barrera);
46          printf("%d paso barrera en etapa %d.\n",id,i);
47      }
48      return NULL;
49  }
50

```

Figura 3: foto del código main.c 1

```

51
52  /*
53  Funcion principal. Aqui se pide al usuario el numero de etapas y de hebras,
54  para luego ser creadas, ademas de crear un array de struct con los datos
55  */
56  int main(int argc, char* argv[]){
57      srand(time(NULL));
58      int N;
59      int E;
60      printf("Numero de hebras y Numero de Etapas:\n");
61      scanf("%d %d", &N, &E);
62
63      barrera_t b;
64      barrera_init(&b, N);
65
66      dHebra datos[N];
67      for(int i=0; i<N; i++){
68          datos[i].id = i;
69          datos[i].E = E;
70          datos[i].b = &b;
71      }
72
73      pthread_t hebras[N];
74      for(int i=0; i<N; i++){
75          pthread_create(&hebras[i], NULL, trabajo_hebra, &datos[i]);
76      }
77
78      for (int i = 0; i < N; i++) {
79          pthread_join(hebras[i], NULL);
80      }
81      barrera_destroy(&b);
82      return 0;
83  }

```

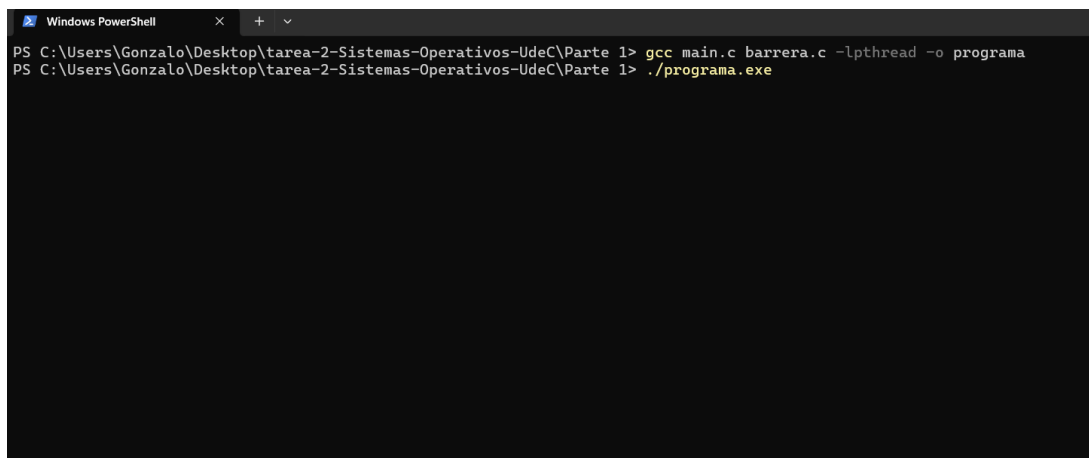
Figura 4: foto del código main.c 2

Aquí las hebras trabajarían con la función `trabajo_hebra`, donde pasaran por el monitor, y tras terminar, en otro ciclo `for` y con la función `pthread_join` se va terminando una por una cada hebra creada. Ya por ultimo, se llama a la función `barrera_destroy` para finalizar con el mutex y cond.

3.4. Prueba y Resultados

Tras finalizar el código, solo nos queda probarlo, para compilar el código, si se usa el compilador gcc, se debe escribir la siguiente linea en la terminal:

`gcc main.c barrera.c -lpthread -o programa` y para ejecutar `./programa.exe`



Numero de hebras y Numero de Etapas:

```
5 4
0 esperando en etapa 0.
2 esperando en etapa 0.
1 esperando en etapa 0.
4 esperando en etapa 0.
3 esperando en etapa 0.
3 paso barrera en etapa 0.
3 esperando en etapa 1.
1 paso barrera en etapa 0.
1 esperando en etapa 1.
2 paso barrera en etapa 0.
2 esperando en etapa 1.
0 paso barrera en etapa 0.
0 esperando en etapa 1.
4 paso barrera en etapa 0.
4 esperando en etapa 1.
4 paso barrera en etapa 1.
4 esperando en etapa 2.
1 paso barrera en etapa 1.
1 esperando en etapa 2.
3 paso barrera en etapa 1.
3 esperando en etapa 2.
2 paso barrera en etapa 1.
2 esperando en etapa 2.
0 paso barrera en etapa 1.
0 esperando en etapa 2.
0 paso barrera en etapa 2.
0 esperando en etapa 3.
1 paso barrera en etapa 2.
1 esperando en etapa 3.
4 paso barrera en etapa 2.
4 esperando en etapa 3.
```

```
3 paso barrera en etapa 2.
3 esperando en etapa 3.
2 paso barrera en etapa 2.
2 esperando en etapa 3.
2 paso barrera en etapa 3.
4 paso barrera en etapa 3.
3 paso barrera en etapa 3.
1 paso barrera en etapa 3.
0 paso barrera en etapa 3.
```

En las imágenes anteriores vemos primero como compilamos y ejecutamos el código, luego ingresamos los datos $N=5$ y $E=4$, y tras presionar enter, se nos muestra el como las 5 hebras (con Id's desde el 0 al 4) llegan a la barrera en la etapa 0 y se quedan esperando, y cuando llegan todas (en este caso, la ultima en llegar fue la hebra 3), cada hebra imprime que ya paso la

barrera en la etapa 0, y pasan a la siguiente etapa. Así hasta llegar a la etapa 3 (como $E=4$, la primera etapa se considera la 0, y la última la 3). Tras pasar esta etapa, las hebras de terminan y acaba la ejecución.

Aquí podemos comprobar que la barrera si es re-utilizable, ya que con la misma, no se genero un error al volver a utilizarla cuando las hebras pasaron de la etapa 0 a la etapa 1, y así etapa por etapa.

Con la realización de este código podemos apreciar la importancia del uso de monitores como esta barrera primitiva o de más avanzados para, no solo para proteger variables globales de que sean cambiadas por cualquier hebra sin ningún cuidado, si no también para poder sincronizar las hebras para que todas puedan seguir a la vez y que ninguna termine antes que la otra.

4. Parte II: Simulador simple de Memoria Virtual

Esta segunda parte, se escribió un código el cual es un simulador secuencial de memoria virtual, el cual procese trazas de páginas virtuales, que soporte un algoritmo de reemplazo Reloj y que pueda medir y reportar el número de fallos de página y la tasa de fallos para distintos parámetros de configuración. El simulador, **sim**, toma y lee una dirección virtual (Que puede ser en decimal o 0xHEX), la traduce a una dirección física, con una paginación simple de marcos físicos de página. Este simulador lo dividimos en los siguientes archivos:

4.1. **funcion.h**

Este archivo define prototipos de las funciones que utilizaremos a lo largo del simulador. En este caso definimos cuatro: **traduce_Hexa**, **traduce_Bi**, **create_Mask** y **lee_direccionV**.

4.2. **funcion.c**

Aquí están las funciones previamente definidas. Estas funciones son auxiliares y destinadas para traducir las direcciones de memoria entregadas, separando estas en el número de pagina y en el offset (desplazamiento).

- **traduce_Hexa**: Esta función tiene como fin el traducir un número Hexadecimal (que le pasamos a la función como una cadenas de texto) a su equivalente decimal. Esta función primero detecta si el argumento entregado empieza con "0x." o con "0X", luego comienza la traducción desde el segundo caracter del argumento, y aprovechando el valor de caracteres numéricos de la tabla ASCII, se traducen a su equivalente entero, tomando en cuenta que en los valores hexadecimales, si un valor es mayor que 9, toma letras que equivalen a ciertos valores (A=10,B=11,C=12,D=13,E=14,F=15). La función retorna un valor entero.
- **traduce_Bi**: Funciona de igual forma de la funcion **traduce_Hexa**, pero en vez de traducir un hexadecimal a decimal, traduce un binario a decimal. Esta función fue añadida por una confusión.
- **create_Mask**: Esta función genera una máscara de bits para extraer el **offset** o desplazamiento dentro de la página. Para ello, simplemente resta 1 al valor **page_size**,

construyendo así un patrón binario con los bits menos significativos en 1. Retorna el valor de dicha máscara.

- **lee_direccionV**: Esta función interpreta una dirección virtual entregada como cadena, separándola en su número de página virtual (**npv**) y su desplazamiento (**offset**). Primero convierte el valor usando *traduce_Hexa()*, luego valida que no sea negativo y que **page_size** sea potencia de 2. Después calcula cuántos bits corresponden al offset, crea la máscara con *create_Mask()*, y extrae ambos componentes mediante operaciones a nivel de bit (**offset** = **valor_dv** & **mask** y **npv** = **valor_dv** ~ **b**). Finalmente entrega **npv** y **offset** mediante punteros.

4.3. sim.h

Al igual del otro archivo ".h", este archivo contiene las definiciones principales relacionadas con el archivo **sim.c**, donde es la base del simulador de manejo de memoria virtual por paginación. En este se incluyen dos struct, uno para las entradas de tabla de páginas (Page Table Entry), donde se tienen variables como el número de página virtual (int npv), una variable para comprobar si la página esta cargada o no en un marco (bool valido) y una variable que sirve de referencia que se utilizara para el algoritmo de reemplazo de página (bool referencia). El otro struct es para el simulador (sim), contiene todas las variables necesarias para el correcto funcionamiento del simulador. Después se declaran tres funciones que se crearan más adelante: **inicializa_sim**, **sim_procesar_dv** y **sim_destruir**.

4.4. sim.c

Esta es el archivo principal del simulador. En este se implementa las funciones previamente declaradas, que permiten el manejo de memoria virtual y del reemplazo de páginas por medio de un **algoritmo de reloj (Clock Algorithm)**. El archivo cuentan con las siguientes funciones:

- **calcular_b**: Aquí se calcula el número de bits necesarios para representar el tamaño de una página, que debe ser una potencia de 2. si el int n que se le entrega al principio es menor o igual a 0, devuelve 0. En caso contrario. devuelve el resultado redondeado de un logaritmo base 2 de n.
- **buscar_npv**: Esta función se encarga de buscar en la tabla de marcos, si la página virtual (que aquí es la variable npv) se encuentra o no cargada en la memoria física.
- **inicializa_sim**: Como se puede intuir por el nombre, esta función se encarga de inicializar los principales valores del simulador. En primer lugar, se cargan los valores iniciales de las variables, varios son dados por medios de argumentos entregados al llamar la función y otros solo se inicializan en cero. Posterior a eso, para **tabla_marcos** se reserva memoria dinámica por medio de la función **calloc()**. Y por ultimo, se añade un manejo en caso de que uno de los procesos anteriores de error.
- **clock_reemplazo**: Aquí se implementa el algoritmo de reemplazo de reloj. Primero, se recorre circularmente la tabla de marcos usando reloj_puntero. Si una página

tiene el bit de referencia en true, se cambia a false (segunda oportunidad). Si una página ya estaba con el bit de referencia en false, entonces se selecciona como víctima, se invalida y ahí termina la búsqueda. Luego, el puntero avanza circularmente para continuar desde esa posición en futuras búsquedas.

- **sim_procesar_dv**: Esta función procesa una dirección virtual y determina si la página solicitada ya está cargada en algún marco de memoria. Primero comprueba si el **npv** se encuentra en la tabla: si está presente, ocurre un *HIT* y únicamente se actualiza el bit de referencia. Si no está, se produce un *FALLO*, se incrementa el contador de fallos de página y se decide dónde ubicar la nueva página. Si quedan marcos libres se utiliza uno; en caso contrario, se invoca al algoritmo de reemplazo **CLOCK** para seleccionar una víctima. Después se almacena el nuevo número de página en el marco escogido, se marca la entrada como válida, y se construye la dirección física combinando el número de marco con el **offset**. Si el modo **-verbose** está activo, la función muestra por pantalla información detallada de cada acceso.
- **sim_destruir**: Esta función libera correctamente la memoria dinámica reservada durante la simulación. Primero verifica que el puntero hacia **tabla_marcos** sea válido y luego utiliza **free()** para liberar el espacio asignado, dejando el puntero en NULL para evitar accesos futuros a memoria ya liberada.

4.5. main.c

En este archivo, primero al ejecutarse, el main recibe unos argumentos donde se determinará la cantidad de marcos, el tamaño de cada marco de página, opcionalmente **-verbose** y el archivo de trazas que tiene las direcciones virtuales que se van a procesar. Luego abre ese archivo y, línea por línea, lee cada dirección virtual en formato texto. Cada dirección se traduce a número y se separa en número de página virtual (**npv**) y desplazamiento (**offset**) mediante la función **lee_direccionV()**. A continuación se llama a la función **sim_procesar_dv()**, que simula el acceso a memoria utilizando el algoritmo de reemplazo de páginas **CLOCK**, determinando si ocurre un *HIT* o un *FALLO* de página y actualizando los contadores correspondientes. Durante este proceso se va incrementando el número total de referencias. Al finalizar la lectura del archivo de trazas, el programa siempre imprime un resumen general que incluye el número total de referencias procesadas, la cantidad de fallos de página y el porcentaje de fallos respecto al total. Adicionalmente, si el programa se ejecuta con la opción **-verbose**, se mostrará información detallada por cada referencia leída, indicando si se produjo un *HIT* o un *FALLO*, el marco utilizado y la dirección física resultante. Finalmente, independiente del modo de ejecución, se libera la memoria reservada para la simulación antes de terminar.

4.6. Prueba y Resultados

En primer lugar hay que compilar y ejecutar el código, para ello, en la terminal, escribimos lo siguiente:

`gcc -o sim main.c sim.c funcion.c -lm` y para ejecutar: **Caso 1:** `./sim 8 4096 trace1.txt` **Caso 2:** `./sim 8 4096 -verbose trace1.txt`

o Caso 1:

```
PS C:\Users\Gonzalo\Desktop\tarea-2-Sistemas-Operativos-UdeC\Parte 2> ./sim 8 4096 trace1.txt
Totales:
Referencias: 8192
Fallos de pagina: 1
Tasa de fallos: 0.01%
PS C:\Users\Gonzalo\Desktop\tarea-2-Sistemas-Operativos-UdeC\Parte 2>
```

o Caso 2:

```
Dv:0x71C (npv:0, offset:1820) | Resultado:HIT | Marco usado:0 | Df: 0x71C
Dv:0xB78 (npv:0, offset:2936) | Resultado:HIT | Marco usado:0 | Df: 0xB78
Dv:0x6FA (npv:0, offset:1786) | Resultado:HIT | Marco usado:0 | Df: 0x6FA
Dv:0xB4C (npv:0, offset:2892) | Resultado:HIT | Marco usado:0 | Df: 0xB4C
Dv:0xD74 (npv:0, offset:3444) | Resultado:HIT | Marco usado:0 | Df: 0xD74
Dv:0x1A5 (npv:0, offset:421) | Resultado:HIT | Marco usado:0 | Df: 0x1A5
Dv:0x7C9 (npv:0, offset:1993) | Resultado:HIT | Marco usado:0 | Df: 0x7C9
Dv:0x769 (npv:0, offset:1897) | Resultado:HIT | Marco usado:0 | Df: 0x769
Dv:0xB96 (npv:0, offset:2966) | Resultado:HIT | Marco usado:0 | Df: 0xB96
Dv:0x257 (npv:0, offset:599) | Resultado:HIT | Marco usado:0 | Df: 0x257
Dv:0xBD8 (npv:0, offset:3032) | Resultado:HIT | Marco usado:0 | Df: 0xBD8
Dv:0xA48 (npv:0, offset:2632) | Resultado:HIT | Marco usado:0 | Df: 0xA48
Dv:0xEA4 (npv:0, offset:3748) | Resultado:HIT | Marco usado:0 | Df: 0xEA4
Dv:0xA14 (npv:0, offset:2580) | Resultado:HIT | Marco usado:0 | Df: 0xA14
Dv:0x449 (npv:0, offset:1097) | Resultado:HIT | Marco usado:0 | Df: 0x449
Dv:0xFE5 (npv:0, offset:4069) | Resultado:HIT | Marco usado:0 | Df: 0xFE5
Dv:0x4D2 (npv:0, offset:1234) | Resultado:HIT | Marco usado:0 | Df: 0x4D2
Dv:0x163 (npv:0, offset:355) | Resultado:HIT | Marco usado:0 | Df: 0x163
Dv:0x5F6 (npv:0, offset:1526) | Resultado:HIT | Marco usado:0 | Df: 0x5F6
Dv:0x3F5 (npv:0, offset:1013) | Resultado:HIT | Marco usado:0 | Df: 0x3F5
Dv:0x402 (npv:0, offset:1026) | Resultado:HIT | Marco usado:0 | Df: 0x402
Dv:0x327 (npv:0, offset:807) | Resultado:HIT | Marco usado:0 | Df: 0x327
Dv:0xF2F (npv:0, offset:3887) | Resultado:HIT | Marco usado:0 | Df: 0xF2F
Dv:0xC6A (npv:0, offset:3178) | Resultado:HIT | Marco usado:0 | Df: 0xC6A
Dv:0x2EC (npv:0, offset:748) | Resultado:HIT | Marco usado:0 | Df: 0x2EC
Dv:0xDCA (npv:0, offset:3530) | Resultado:HIT | Marco usado:0 | Df: 0xDCA
Dv:0xE53 (npv:0, offset:3667) | Resultado:HIT | Marco usado:0 | Df: 0xE53
Dv:0x3C6 (npv:0, offset:966) | Resultado:HIT | Marco usado:0 | Df: 0x3C6
Dv:0x6A8 (npv:0, offset:1704) | Resultado:HIT | Marco usado:0 | Df: 0x6A8
Dv:0x443 (npv:0, offset:1091) | Resultado:HIT | Marco usado:0 | Df: 0x443
Dv:0x689 (npv:0, offset:1673) | Resultado:HIT | Marco usado:0 | Df: 0x689
Dv:0x48D (npv:0, offset:1165) | Resultado:HIT | Marco usado:0 | Df: 0x48D
Dv:0x172 (npv:0, offset:370) | Resultado:HIT | Marco usado:0 | Df: 0x172
Dv:0x315 (npv:0, offset:789) | Resultado:HIT | Marco usado:0 | Df: 0x315
Dv:0xE43 (npv:0, offset:3651) | Resultado:HIT | Marco usado:0 | Df: 0xE43
Dv:0x422 (npv:0, offset:1058) | Resultado:HIT | Marco usado:0 | Df: 0x422

Totales:
Referencias: 8192
Fallos de pagina: 1
Tasa de fallos: 0.01%
```

5. Conclusiones

Por medio del desarrollo de ambos códigos, tanto de la barrera primitiva como del simulador de memoria virtual, hemos podido no solo comprender la importancia del manejo de memoria y la utilización de monitores, sino también apreciar a nivel de código la implementación de estructuras de control de concurrencia y la gestión eficiente de recursos, y nos permite comprender cómo estos conceptos se aplican en sistemas reales, optimizando el rendimiento y evitando errores de concurrencia y memoria.